

Step 1: Set up your computer

Open a terminal and install Python packages:

```
pip install numpy librosa sounddevice torch torchaudio scikit-learn matplotlib
```

This needs to run on **your own machine**, not in a sandbox — because Step 2 needs your actual microphone. No GPU needed, a laptop CPU is fine.

Step 2: Pick your wake word

Something 2–4 syllables long works best (too short = false triggers, too long = annoying to say).

Example: "Hey Nova" or your own made-up phrase.

Step 3: Record your data

This is the biggest and most important chunk of work — a wake word detector is only as good as the data you feed it.

- **Positive samples:** 50–100 recordings of you saying the wake word. Vary distance from mic, tone (normal/quiet/excited/tired), and spread recordings across a few different days.
- **Negative samples:** 150–300+ recordings of *everything else* — silence, background noise, you talking normally, and especially words that sound similar to your wake word. Negatives should outnumber positives by at least 3:1, because in real life the system hears way more non-wake-word audio than wake-word audio.

I gave you a small Python recording script earlier that automates this (records a clip, saves it, waits for you to hit Enter, repeats). You just run it and talk into your mic.

Step 4: Turn audio into features (MFCCs)

Raw audio is just a wall of numbers — not something a neural net can learn from directly. You convert each clip into an MFCC (a compact numeric fingerprint of the sound, based on how human hearing perceives pitch). librosa does this in one function call. Output is a small 2D grid per clip — that's what your model actually trains on.

Step 5: Build and train a small CNN

Since the MFCC grid looks like a tiny image, a small Convolutional Neural Network works well. You split your data into train/validation/test sets, then run a training loop (predict → measure error → adjust weights) for ~30 epochs. Takes a few minutes on CPU.

Step 6: Evaluate honestly

Don't just check accuracy — check **precision** (how often "detected" was correct) and **recall** (how often you actually said it and it caught you). A lazy model that never triggers can look "accurate" while being useless.

Step 7: Wire it into real-time listening

Once trained, you run a script that continuously listens through the mic in overlapping ~1.5 second windows, runs each window through the model, and prints "Wake word detected!" when it fires.

Step 8: Iterate

Your first model will not be great. Test it live, notice what breaks (too far from mic? confuses a similar word?), record more targeted data for that weakness, retrain. Repeat 2–3 times.